四川农业大学工作相关

教务处

处章

雅安校区周一至周二10:00-16:00，成都校区周三至周五10:00-16:00

成都校区1-104相关

工作wifi:TP-LINK_104，密码:86290588

摄像头wifi:jwc104，密码:CDjwc0588

教务处处长:曹三杰，电话18608352183，成都校区1教212

教务处副处长:李伟，电话13981733950，成都校区1教214

教务二科科长:李振永，电话18780023094，成都校区1教104

教务处办公室:王龙，电话13980451730，成都校区1教215

教务处办公室:刘春娟，成都校区1教215

教务处副科长:宗小兰，成都校区1教208

徐大武账号相关

工号73294

一卡通账号密码&校园网账号密码

73294

131420

统一身份认证账号密码&教务处账号密码

73294

Aimer1x@@

全国教师管理信息系统信息账号密码

51052119960501799X

Aimer1x@@

工作时间2024_09_20

周1至周4

8:30-12:00

14:30-18:00

周五&节假日前一天

8:30-12:00

14:30-16:30

会议信息2024_09_27

雅安交流会2024_09_26

参与人：徐大武、李伟（教务处处长）、曾昌伟（本科教学运行）

曾昌伟：20年来川农工作，08年建教务处系统使用至今；浙江大学的教务管理系统值得学习

周例会2024_09_23

第4周运行例会2024_09_23_11:15_12:15

发言人:张亮

内容：

前三周工作：新生注册（学信网）、火车票优惠注册、新生学生证注册下发

学分预警

发言人:向（课业安排）

内容：

前三周工作：重修选课、课表调整、课程大纲教案收集

接下来三周安排：

发言人:曾昌伟

内容：

前三周工作：四六级报名信息管理、云平台试卷材料归档

接下来三周安排：

发言人:（考试）

内容：

前三周工作：成绩协查、学分置换

接下来三周安排：免修库整理

发言人:刘春娟（教材）

内容：

前三周工作：教程发放、涉密教程排查

接下来三周安排：重修复学等教材补发/退订

发言人:李振永（信息）

内容：

前三周工作：系统维护和研发、智慧教室建设、机房维护、协同育人评价管理系统

接下来三周安排：重修复学等教材补发/退订

发言人:李处（教务运行）

内容：

前三周工作：

接下来三周安排：本科教学审核评估报告、重视教育教学改革、本科教学持续稳定进行

第7周运行例会2024_10_14_16:02

发言人：曾昌伟（考试）

四六级编排、上报完成，成绩单资料整理完成，

发言人：曹冉（成绩）

评优评奖、推免、成绩修改

发言人：刘春娟（教材）

完成工作

教材补发、退订、

后续安排：

教材清退、对账

发言人：李振永（信息）

虚拟仿真教室维护

项目开发

发言人：徐大武（信息）

用户的安全认证基本完成

后续工作：

新系统的迁移和后续开发

发言人：李伟（运行）

教学运行费审核（李振永）

本科审核评估10.23启动

按阶段调阅资料、补充调阅资料、专家线下评估（参观）、

线上评估：评鉴办

综合管理组：曹处、王龙

听课看课组：康波、周军

座谈访谈组：学生处谢建平，访谈学生、教师

资料调阅组：李伟、

后勤保障组：林瑶、王欣怡

在线巡课系统：90多间教室装备摄像头，进行到课率等督导

第11周运行例会2024_11_11_16:00

发言人：徐大武

教务系统平台搭建完毕

第一个功能成绩可视化基本做完了

第二个超级查询功能也基本做完了

教务系统的数据越来越多，这个超级查询可以一个接口解决所有想查而我们教务处又有的信息，但是各个岗位的查询权限分配需要细化

上周和信教中心对接了，项目流程走了一遍，把权限分配和网络安全再详细测试后，就可以部署到信教中心提供的容器资源里，开放内网甚至外网的访问

然后以上的事情完成后，就开始做排课的优化

第13周运行例会2024_11_26_11:00

发言人：张亮（学籍）

学籍学信网互通注册

学籍清退

发言人：刘春娟（教材）

教材费结算、准备

发言人：徐大武

排课系统建设，算法和模拟数据都测试完了

现在是用上学期的排课数据做真实的数据测试，并根据真实数据来修正系统

排课牵涉到很多的表数据，比如开课任务，教师，教室等这些表，现在在梳理开课任务表，表里有很多的字段，我需要搞清楚它的含义和排课的关系，所以现在就是在梳理数据和处理数据阶段

第16周运行例会2024_12_16_17:00

发言人：徐大武

排课系统建设，现在基本完成了，处在修正阶段，在优化速度和准确性，已经设置了部分规则也测试完了，现在逐步的把剩余的排课规则加上去并修正，接下来就是补充规则

根据实际数据，大的算法框架已经换了3个了

工作日常2024_10_18

智慧教室启用发布程序2024_10_18

校区，教学单位，姓名，工号，手机号

使用金山文档收集表单数据

后勤处

成都校区报修:(028)86292117

人事处

成都校区办公室：（028）86293061

党委办公室

信息系统建设2024_09_20

一、新教务系统建设2024_09_20

开发语言

使用python语言为基底，适应业务不断变化的需求

后端框架

使用FastAPI作为后端框架，满足性能需求和业务拓展需求

后端数据库sqlserver2022

用户名：sa

密码：scau_jwc_sqlserver

数据库查询原则

有两种方案

- **一、一次查询，后续在Python中处理**

一次从数据库查询获得数据，然后后续每次数据处理都通过python语言体系处理第一次查询出的完整庞大数据

优点：

1. **减少数据库查询次数**：一次性获取所有数据，减少了与数据库的交互次数，这对于网络延迟较高的环境非常有利。
2. **灵活性**：在内存中操作数据，可以使用Python丰富的数据处理库（如Pandas）进行复杂的分析和处理。
3. **适合小数据集**：如果数据量较小，一次性加载到内存中处理是比较高效的。

缺点：

1. **内存消耗**：如果数据集较大，可能会导致内存不足或性能下降。
2. **初始加载时间**：一次性加载大量数据可能会导致初始查询时间较长。

- **二、多次查询，每次只获取需要的数据**

多次从数据库查询获得数据，然后后续每次数据处理都通过python处理查询出的小部分数据

优点：

1. **内存利用**：只在内存中保留必要的数据，适合处理大数据集，减少了内存消耗。
2. **即时数据**：每次查询获取最新的数据，适用于数据变化频繁的场景。

缺点：

1. **查询次数多**：每次操作都需要查询数据库，增加了与数据库的交互次数，可能导致性能下降。
2. **网络延迟**：频繁的数据库查询可能导致网络延迟影响整体性能，尤其是在远程数据库连接时。

选择哪种方案取决于具体的使用场景：

- 如果数据量小且处理复杂度高，方案一更为合适。
- 如果数据量大或需要处理的数据频繁变化，方案二更为合适。

本项目为了降低服务器查询压力，优先采用第一种方案

数据库字符编码

统一为UTF-8格式

数据表

用户表

新增哈希密码字段，用于新系统的登录密码验证

成绩表

学生成绩数据共2808400条

二、Docker

启动docker-compose.yml文件服务

cd到文件所在根目录

```
docker-compose up
```

停止服务

```
docker-compose down
```

使用Docker部署项目

自定义网路

```
docker network create docker_bridge_network
```

数据库sqlserver

拉取镜像

```
docker pull mcr.microsoft.com/mssql/server
```

启动容器

```
docker run --name sqlserver2022 -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=sicau_jwc_sqlserver123" -p 1433:1433 --restart always -v C:\xudawu\DockerData2024_09_29\SqlServer_2024_10_30:/var/opt/mssql/data -d mcr.microsoft.com/mssql/server
```

数据库名称：jxgl_all

用户名：sa

密码：sicau_jwc_sqlserver123

数据库postgres

启动容器

```
docker run --name hello_postgres -e POSTGRES_PASSWORD=docker_postgres --restart always -v C:\xudawu\DockerData2024_09_29\postgres_2024_10_31:/var/lib/postgresql/data -p 5432:5432 -d postgres
```

用户名：postgres

密码：docker_postgres

挂载路径：C:\xudawu\DockerData2024_09_29\postgres_2024_10_31

数据库字符编码格式：UTF-8

数据库mysql

启动容器

```
docker run --name mysql910 -e MYSQL_ROOT_PASSWORD=sicau_jwc_mysql123 -p 3306:3306 -v C:\xudawu\DockerData2024_09_29\mysql_2024_11_07:/var/lib/mysql --restart always -d mysql:9.1.0
```

web全栈

Dockerfile文件

```
# 使用官方的 Python镜像作为基础镜像
FROM python:3.12.6

# 设置工作目录
WORKDIR /sicau_jwc

# 安装 unixODBC 和sqlserver驱动，并清理缓存
RUN apt-get update && \
    apt-get install -y curl && \
    curl https://packages.microsoft.com/keys/microsoft.asc | apt-key add - && \
    curl https://packages.microsoft.com/config/ubuntu/20.04/prod.list > /etc/apt/sources.list.d/mssql-release.list && \
    apt-get update && \
    ACCEPT_EULA=Y apt-get install -y msodbcsql18 mssql-tools unixodbc-dev && \
    rm -rf /var/lib/apt/lists/*

# 安装 unixODBC 和 PostgreSQL ODBC 驱动
# RUN apt-get update && \
#     apt-get install -y unixodbc unixodbc-dev odbc-postgresql && \
#     rm -rf /var/lib/apt/lists/*


# 将 requirements.txt 文件复制到工作目录
COPY requirements.txt .

# 安装项目依赖
# 在容器中执行 pip install 命令，安装 requirements.txt 中列出的所有依赖包。
# --no-cache-dir 参数用于避免在安装时缓存包，从而减小镜像体积。
# -i https://pypi.tuna.tsinghua.edu.cn/simple使用清华大学的镜像源
```

```
RUN pip install --no-cache-dir -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple

# 将项目代码复制到工作目录
# 第一次参数是相对于Dockerfile的路径，第二次参数是相对于WORKDIR的路径
COPY . /sicau_jwc

# 创建数据存储目录
RUN mkdir -p /sicau_jwc/data

# 声明容器在运行时将使用的端口
EXPOSE 8000

# 启动 FastAPI 应用，使用 uvicorn 作为 ASGI 服务器
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

打包项目

cd到项目根目录

```
docker build -t sicau_jwc_image:v20241113 .
```

docker启动容器命令

```
docker run --restart always --name sicau_jwc_container -p 8000:8000 -v C:\xudawu\DockerData2024_09_29\scau_jwc_2024_11_01:/sicau_jwc/data -d sicau_jwc_image:v20241113
```

导出docker项目

```
docker save -o sicau_jwc.tar sicau_jwc_image:v20241113
```

导入docker项目

```
docker load -i sicau_jwc.tar
```

标记镜像

```
docker tag sicau_jwc_image:latest rhythmcc/sicau_jwc:v20241113
```

推送镜像

```
docker push rhythmcc/sicau_jwc:v20241113
```

测试和运行docker项目

```
docker build -t test_image .
docker run  --name test_container -p 8000:8000 -d test_image
```

数据库可视化管理工具

navicat17

Stirling-PDF

pdf处理工具

dockerhub：https://hub.docker.com/r/frooodle/s-pdf

github：https://github.com/Stirling-Tools/Stirling-PDF

docker部署

```
docker pull frooodle/s-pdf
```

启动容器

```
docker run --name s_pdf_container --restart always -p 8080:8080 -d
frooodle/s-pdf
```

三、信教中心分配容器信息

ip

```
10.255.252.184
```

vpn账号信息

同统一身份认证

用户名：73294

密码：Aimer1x@@

堡垒机账号信息

同统一身份认证

用户名：73294

密码：Aimer1x@@

容器资源账号信息

用户名：jw73294

密码：Aimer1x@@

信教中心私有仓库

网址

```
https://sicau-harbor.sicau.edu.cn/
```

用户名：73294

密码：Aimer1x@@

登录私有仓库

```
docker login https://sicau-harbor.sicau.edu.cn
```

标记镜像

```
docker tag sicau_jwc_image:v20241219 sicau-harbor.sicau.edu.cn/sicau_jwc/repository:v20241219
```

push镜像

```
docker push sicau-harbor.sicau.edu.cn/sicau_jwc/repository:v20241219
```

s-pdf的推送仓库

```
docker tag frooodle/s-pdf:latest sicau-harbor.sicau.edu.cn/sicau_jwc_spdf/repository:v20241113

docker push sicau-harbor.sicau.edu.cn/sicau_jwc_spdf/repository:v20241113
```

四、本科学生学习成绩可视化2024_09_20

初稿2024_09_17

读取excel文件中的学生成绩数据,按高考成绩排序将学生划分为3类
分别为前1/3,中1/3,后1/3,分别计算每一个学生的平均分,然后按学生所属的类别赋予线条颜色绘制成绩走势图
并做两条标准线,分别为每学年前30%和前70%平均分的标准线

改进建议2024_09_20

每个省份高考分数线不一，川农大概40%的学生是四川，只做四川学生

职教和体育分数特殊需要单独考虑

技术支持2024_10_01

fastapi后端进行数据处理、数据呈现于图像plotly、将图像渲染到前端html

数据流程2024_10_25

成绩折线图开放可选参数以下5个

- 1.班级名，通过筛选学院再筛选班级，默认数据库查询的第一个
- 2.课程类型，默认必修
- 3.图像高度，默认600px
- 4.图像宽度，默认1000px
- 5.y轴刻度间隔，默认5

自动计算匹配以下参数

- 1.学院
- 2.班主任
- 3.班级人数
- 4.学号
- 5.学期

数据处理

筛选框数据

1. 页面加载时查询所有学院和学院对应的班级，填充到下拉选择框中
2. 用户选择学院和班级之后，再自动查询班级在成绩表中有成绩的课程性质
3. 用户选择学院、班级、课程性质之后，提交筛选信息到后端进行数据处理

图表数据

1. 根据班级和课程性质，获取所有学期，将学期进行排序
2. 分别将每个学期班级的学生成绩按学号分组，再按分组计算平均分，前30%和前70%的分位点
3. 有多少学期就计算多少学期

其他数据

- 学院:学院通过成绩分班表获得，只保留成绩表中有成绩的班级所属的学院
- 班级:班级通过成绩表获得，只保留成绩表中有成绩的班级
- 课程类型:通过成绩表获得，只保留有成绩的课程类型
- 学期：自动匹配学期时，需要满足课程类型有的学期，而不能用班级的所有学期

特殊数据处理

设置查询班主任为空时设置为'None'，如动科院的水产201704，没有班主任

相关数据库表2024_10_23

成绩表

表名：成绩

学生成绩来源

班级有成绩的所有学期来源

成绩分班表

表名：成绩分班

班级和学院的对应来源

班级数量3201个

不是每个班级都有成绩，仅保留有成绩的班级1930个

班级表

表名：班级

班级和班主任的对应来源

五、新排课系统2024_09_20

川农教学排课现状2024_09_27

- 课程与教师对应关系为多对多： 一个教师可以上多门课，一门课可以由多个老师授课
- 排课压力不应施加太高：不连堂排课，中午可以视为重置的时间节点
- 教室排课指定：某些教室只能排理论课或者实验课
- 教室容量：教室只能容纳选课的有限数量学生
- 四川农业大学2.4k班级；900间教室；一学期500授课教师；一学期6500门授课；3.8w学生
- 教务管理系统数据库大小17GB，表数量994

当前川农排课流程

1. 培养计划获得课程信息，培养方案和学籍生成教学任务（上课计划）

2. 分和班下发学院填报教师信息
3. 系统排课
4. 内部检查调整再下发学院任课教师调整排课信息

当前川农排课相关数据表

表名

- 开课任务：授课计划，绑定教师和学生，对应排课流程1，混教（理论课）、选课（公选课）、课程编号（从培养计划来的课程编号）、编号（指教学班（选这门课的学生形成的班）编号）、周次（共30位，1代表要排课的周，0代表不排课）、周次1（排课始末周）
- 教室：混教（理论课）、选课（公选课）可以选择的教室，间1考容（间隔1个座位考试的容量），是否排课（若为否则不安排排课）
- sy实验室：实验课可以选择的实验教室
- 开课限制课程：课程时间限制
- 开课限制教师：教师不想上课的时间，时间限制（1-1代表不想上课时间为星期1第1讲，0代表没选）
- 开课任务_排课：本学期已排好的课

排课规则

- 列名课程体系为慕课的不排、列名课程性质为实践教学的不排
- 教学计划：从培养方案生成的课程信息

遗传算法2024_09_23

遗传算法概念2024_09_23

遗传算法是一种模拟自然选择过程的启发式优化方法，适用于求解复杂的离散优化问题，遗传算法引入了一定程度的随机性以增加搜索的多样性，从而提高找到全局最优解的可能性。在排课问题中，需要在多个时间段和教室之间安排一系列课程，并尽量满足一系列约束条件（如避免冲突、教师可用性、教室容量等）。以下是遗传算法在排课问题中的基本工作流程：

1.初始种群生成 (Initialization)

- **个体表示 (Chromosome Representation)**: 在排课问题中，每个个体（染色体）表示一种排课方案。其中染色体上的每个基因元素表示某门课程在某个时间段和教室的安排。
- **种群初始化 (Population Initialization)**: 生成一定数量的随机排课方案，作为初始种群。

2.适应度评估 (Fitness Evaluation)

- 为每个个体计算一个适应度值（Fitness Value），表示该排课方案的优劣。适应度值的计算根据以下几个方面：
  - 是否存在时间冲突（如同一时间段安排了多门课程在同一个教室）。
  - 教师的排课均衡（如教师是否安排多次或未安排课程）。
  - 课程优先级（如优先级高的课程是否排课）。
  - 禁止排课时间（如课程不可以安排在某一时间段）。

3.选择 (Selection)

- 根据适应度值选择父代个体，优良个体被选择的概率更高。常见的选择方法包括轮盘赌选择 (Roulette Wheel Selection)、锦标赛选择 (Tournament Selection) 等。这里以适应度排名只选取适应度最高部分几位作为父母

4.交叉 (Crossover)

- 从选择的父代个体中随机挑选两个进行交叉操作，生成新的后代个体（即新的排课方案）。常用的交叉方法有：
  - 单点交叉 (Single-Point Crossover)：在染色体的某一位置切分，交换两部分。
  - 多点交叉 (Multi-Point Crossover)：在多个位置切分，并交换切分的片段。

5. 变异 (Mutation)

- 对后代个体进行小概率的随机变异操作，以增加种群的多样性。对于排课问题，变异表现为随机调整某门课程的时间和教室安排。

6. 新一代种群形成 (New Population)

- 由选择、交叉和变异操作生成的新个体和适应度最高的父代个体构成新一代种群。（为了维持种群质量，将前一代种群中最优的个体（精英保留）直接保留到新一代。）

7. 终止条件 (Termination Condition)

- 算法通常在满足以下条件之一时终止：
  - 达到了最大迭代次数。
  - 适应度值没有显著提升（收敛）。
  - 找到了满足所有约束条件的最优排课方案（即适应度值达到设置的阈值）。

8. 解的提取 (Solution Extraction)

- 从最终种群中选择适应度值最高的个体，作为最终的排课方案。

优点

- **灵活性**：可以处理多种复杂的约束条件。
- **全局优化能力**：避免陷入局部最优解。
- **适应性强**：可以适用于不同规模和复杂度的排课问题。

缺点

- **计算成本高**：对于大型问题，算法收敛速度较慢。
- **参数调节复杂**：种群大小、交叉概率、变异概率等参数的选择对算法性能影响较大。

整体评价

遗传算法在排课问题中的应用具有较高的有效性和灵活性，实际实现中需要根据具体问题对适应度函数和参数进行调优。

遗传算法具体实现2024_09_24

下面结合代码细节来介绍遗传算法的各个步骤。

1. 课程、课程表和学生类的定义

- `Course` **类**：定义课程的基本属性，包括课程名称、教师名称、持续时间、优先级和不可用时间段。
- `Student` **类**：表示学生，包含学生姓名和选课列表，以及已上课的时间段。上课时间段根据排课安排自动生成
- `Schedule` **类**：维护一个课程表，包含课程、时间段和教室的信息。初始化时，所有时间段和教室组合都是空的。

2. 适应度评估 (Fitness Function)

```
fitness
```

方法通过以下方式评估一个排课方案的适应度：

- **课程冲突**：同一课程在同一时间安排到多个教室将扣分。
- **教师安排**：教师被合理安排课程的次数，过多或过少都会影响适应度。
- **学生冲突**：学生不能在同一时间上多门课，冲突时会大幅降低适应度。
- **教师不可用时间**：如果课程安排在教师不可用的时间，适应度会降低。教师不可用时间如'2-2,3-2'

用','分割时间，再用'-'分割时间为星期几和第几讲数，再默认填充0作为周次，代表所有周

3. 变异 (Mutation)

- `mutate`方法随机选择一个时间段和教室组合，然后随机更改此时间和教室安排的课程。增加种群基因的多样性，探索更多的可能性。

4. 交叉 (Crossover)

- `crossover`方法从几个父代排课方案中生成两个子代。它随机选择一个交叉点，然后交换交叉点后的部分课程，生成新的排课方案。这有助于保留良好的排课特性，并组合不同个体的优点。

5. 遗传算法流程

1. **初始化种群**：生成一定数量的随机课程安排作为初始种群。
2. **进化过程**：
    - 计算每个个体的适应度。
    - 保留适应度最高的个体。
    - 随机选择高适应度个体进行交叉和变异，生成下一代。
    - 重复直到找到满足适应度阈值的最佳安排。

遗传算法排课具体代码（python）2024_09_24

```python
'''
Author: xudawu
Date: 2024-09-18 16:39:56
LastEditors: xudawu
LastEditTime: 2024-09-25 11:49:06
'''
import random

class Course:
    def __init__(self, name_str, teacher_str, duration_float, priority_float=1, unavailable_timeslots_list=[]):
        self.name_str = name_str
        self.teacher_str = teacher_str
        self.duration_float = duration_float
        self.priority_float = priority_float
        # 不可用的时间列表
        self.unavailable_timeslots_list = unavailable_timeslots_list if unavailable_timeslots_list else []

class Student:
    def __init__(self, name_str):
        self.name_str = name_str
        self.enrolled_courses_list = []
        # 记录学生的上课时间段
        self.time_slots_list = []

    def enroll(self, course):
```

```python
        self.enrolled_courses_list.append(course)

class Schedule:
    def __init__(self, course_list, timeslots_list, room_list, student_list):
        self.course_list = course_list
        self.timeslots_list = timeslots_list
        self.room_list = room_list
        self.student_list = student_list
        # 时段和教室的组合作为键，课程作为值,课程初始化为None
        self.schedule_dict = {}
        for cur_timeslot_str in self.timeslots_list:
            for cur_room_str in self.room_list:
                self.schedule_dict[(cur_timeslot_str, cur_room_str)] = None

    def fitness(self):
        fitness_float = 0
        course_time_dict = {}
        teacher_assigned_dict = {}

        # 初始化学生冲突时间表为空列表
        for CurStudent in self.student_list:
            CurStudent.time_slots_list = []

        # 初始化冲突信息的列表
        self.conflict_info_list = []

        # 每个教师被安排的次数初始化为0
        for CurCourse in self.course_list:
            teacher_assigned_dict[CurCourse.teacher_str] = 0

        for (cur_timeslot_str, cur_room_str), CurCourse in self.schedule_dict.items():
            # 如果当前时段有课程安排,则进行判断
            if CurCourse is not None:
                # 同一课程同一时间不能安排在多个教室,同一时间同一课程名代表有冲突
                # 在安排中则代表有冲突
                if (cur_timeslot_str, CurCourse.name_str) in course_time_dict:
                    fitness_float -= 3
                else:
                    # 没有在安排表中，记录此次安排
                    course_time_dict[(cur_timeslot_str, CurCourse.name_str)] = cur_room_str
                    # 安排此课,适应度分值加上优先级分值
                    fitness_float += CurCourse.priority_float
                    # 统计教师被安排的次数
                    teacher_assigned_dict[CurCourse.teacher_str] += 1

                    # 学生不能同时上两门及以上课程,进行冲突检测
                    for CurStudent in self.student_list:
                        if CurCourse in CurStudent.enrolled_courses_list:
                            if cur_timeslot_str in CurStudent.time_slots_list:
                                # 有冲突大幅降低适应度
                                fitness_float -= 3
                                # 记录冲突信息
```

```
 self.conflict_info_list.append((CurStudent.name_str, CurCourse.name_str, cur_timeslot_str))

                            # 更新学生的时间段表,增加此上课时间到学生的上课时间表中
                            else:

 CurStudent.time_slots_list.append(cur_timeslot_str)

                # 如果此门课程的安排时间在教师不可以上课的时间列表中，适应度大幅降低
                if cur_timeslot_str in CurCourse.unavailable_timeslots_list:
                    fitness_float -= 3

        # 教师排课均衡性评分
        for course_teacher_name_str, teacher_assigned_count in teacher_assigned_dict.items():
            # 教师没有被分配到课程
            if teacher_assigned_count == 0:
                fitness_float -= 5
            # 教师被分配了大于等于3次课程
            elif teacher_assigned_count >= 3:
                fitness_float -= 2
            # 教师排课次数均衡
            else:
                fitness_float += 1

        # 返回适应度
        return fitness_float

    # 基因变异操作
    def mutate(self):
        # 随机选择一个时间段和教室组合
        random_timeslot_room_tuple = random.choice(list(self.schedule_dict.keys()))
        # 随机选择一门课程
        random_new_course_str = random.choice(self.course_list)
        # 替换该位置的课程
        self.schedule_dict[random_timeslot_room_tuple] = random_new_course_str

    # 基因交叉操作
    def crossover(self, other):
        # 随机选择交叉基因的起点
        crossover_index_int = random.randint(0, len(self.schedule_dict) - 1)
        # 初始化两个子代的基因
        child1_schedule_dict = {}
        child2_schedule_dict = {}

        # 迭代所有时间和房间的键值对
        for i, schedule_key_tuple in enumerate(self.schedule_dict.keys()):
            # 子代1交叉基因起点之前的基因来自本个体,子代2的基因来自其他个体
            if i <= crossover_index_int:
                child1_schedule_dict[schedule_key_tuple] = self.schedule_dict[schedule_key_tuple]
                child2_schedule_dict[schedule_key_tuple] = other.schedule_dict[schedule_key_tuple]
```

```python
            else:
            # 子代1交叉基因起点之后的基因来自其他个体,子代2的基因来自本个体
                child1_schedule_dict[schedule_key_tuple] = other.schedule_dict[schedule_key_tuple]
                child2_schedule_dict[schedule_key_tuple] = self.schedule_dict[schedule_key_tuple]

        # 初始化两个子代类的基因
        Child1Schedule = Schedule(self.course_list, self.timeslots_list, self.room_list, self.student_list)
        Child2Schedule = Schedule(self.course_list, self.timeslots_list, self.room_list, self.student_list)
        # 赋予子代基因
        Child1Schedule.schedule_dict = child1_schedule_dict
        Child2Schedule.schedule_dict = child2_schedule_dict

        # 返回两个新子代基因
        return Child1Schedule, Child2Schedule

    def display(self):
        # 存储未安排课程的教师、学生和教室
        unassigned_teachers = set(teacher.teacher_str for teacher in self.course_list)
        unassigned_rooms = set(self.room_list)
        unassigned_students = set(student.name_str for student in self.student_list)

        for (cur_timeslot_str, cur_room_str), CurCourse in self.schedule_dict.items():
            # 如果课程为非空,则打印课程信息
            if CurCourse is not None:
                # 初始化本门课的选课学生列表
                enrolled_course_student_name_list = []
                # 遍历学生列表
                for CurStudent in self.student_list:
                    # 如果本门课在当前学生选课列表中,则将学生姓名加入选课学生列表
                    if CurCourse in CurStudent.enrolled_courses_list:
                        enrolled_course_student_name_list.append(CurStudent.name_str)
                        # 从未安排列表中移除学生
                        unassigned_students.discard(CurStudent.name_str)
                enrolled_course_student_str = ", ".join(enrolled_course_student_name_list) if enrolled_course_student_name_list else "No student enrolled"

                # 检查教师是否在不可上课时间
                teacher_unavailable = cur_timeslot_str in CurCourse.unavailable_timeslots_list
                availability_str = " | Teacher unavailable during this time" if teacher_unavailable else ""


                # 初始化冲突信息
                conflict_str = ""
                for conflict_info_tuple in self.conflict_info_list:
                    if conflict_info_tuple[1] == CurCourse.name_str and conflict_info_tuple[2] == cur_timeslot_str:
```

```
                        conflict_str = f" | Conflict: {conflict_info_tuple[0]}"

                print(f"{cur_timeslot_str} in {cur_room_str}: {CurCourse.name_str} by {CurCourse.teacher_str} (Priority: {CurCourse.priority_float}) | Enrolled: {enrolled_course_student_str} {conflict_str}{availability_str}")

                # 从未安排列表中移除教师和教室
                unassigned_teachers.discard(CurCourse.teacher_str)
                unassigned_rooms.discard(cur_room_str)
            else:
                print(f"{cur_timeslot_str} in {cur_room_str}: Free")

        # 打印未安排的教师、教室和学生
        if unassigned_teachers:
            print("Unassigned Teachers: " + ", ".join(unassigned_teachers))
        else:
            print("All teachers assigned.")

        if unassigned_rooms:
            print("Unassigned Rooms: " + ", ".join(unassigned_rooms))
        else:
            print("All rooms assigned.")

        if unassigned_students:
            print("Unassigned Students: " + ", ".join(unassigned_students))
        else:
            print("All students enrolled.")


# 初始化种群
def initialize_population(course_list, timeslots_list, room_list, student_list, population_size=100):
    # 种群列表初始化
    population_list = []
    for _ in range(population_size):
        # 实例化一个个体
        SingleSchedule = Schedule(course_list, timeslots_list, room_list, student_list)
        # 对个体的基因进行随机初始化
        for schedule_key_tuple in SingleSchedule.schedule_dict.keys():
             # 随机一个基因(对时间段和教室的组合随机选择一门课)
            SingleSchedule.schedule_dict[schedule_key_tuple] = random.choice(course_list)

        # 将初始化后的个体加入种群列表
        population_list.append(SingleSchedule)

    # 返回初始化的种群列表
    return population_list

# 交叉和变异获得下一代种群
def genetic_evolution(population_list,retention_number_int):

    # 计算适应度,并对种群列表进行倒序排序
```

```python
    population_list.sort(key=lambda x: x.fitness(), reverse=True)

    # 初始化下一代个体列表
    next_generation_list = []
    # 保留最优的部分个体
    for i in range(retention_number_int):
        next_generation_list.append(population_list[i])

    # 选择和交叉,直到达到种群大小
    while len(next_generation_list) < len(population_list):
        # 随机在适应度最高的部分中选择父母
        parent1 = random.choice(population_list[:10])
        parent2 = random.choice(population_list[:10])
        # 对父母基因进行交叉,得到两个子代
        child1, child2 = parent1.crossover(parent2)
        # 将两个子代加入下一代个体列表
        next_generation_list.append(child1)
        next_generation_list.append(child2)

    # 子代和父代有概率变异
    for ChildsChedule in next_generation_list:
        if random.random() < 0.3:
            ChildsChedule.mutate()

    return next_generation_list

if __name__ == "__main__":
    # 示例课程（课程名、任课教师、课时长(单位小时)、优先级(数字越高优先级越高)、不可上课时间）
    course_list = [
        Course(
            name_str="class1", teacher_str='teacher1', duration_float=1,
            priority_float=1, unavailable_timeslots_list=["time2", "time4"]
            ),
        Course("class2", "teacher2", 1, 1, ["time1"]),
        Course("class3", "teacher3", 1, 1),
        Course("class4", "teacher4", 1, 1, ["time3"]),
        Course("class5", "teacher5", 1, 1),
        Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
        Course("class7", "teacher7", 1, 1),
        Course("class8", "teacher8", 1, 1, ["time2"]),
        Course("class9", "teacher9", 1, 1),
        Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
        Course("class11", 'teacher11', 1, 1),
        Course("class12", "teacher12", 1, 1, ["time1"]),
        Course("class13", "teacher13", 1, 1),
        Course("class14", "teacher14", 1, 1, ["time3"]),
        Course("class15", "teacher15", 1, 1),
        Course("class16", "teacher16", 1, 1, ["time5"]),
        Course("class17", "teacher17", 1, 2),
        Course("class18", "teacher18", 1, 2, ["time2"]),
        Course("class19", "teacher19", 1, 3),
        Course("class20", "teacher20", 1, 3, ["time4"]),
    ]

    # 实例时间段和教室
```

```python
    timeslots_list = ["time1", "time2", "time3", "time4", "time5", "time6"]
    room_list = ["Room1", "Room2", "Room3"]

    # 示例学生
    student_list = [
        Student("student1"),Student("student2"),Student("student3"),
        Student("student4"),Student("student5"),Student("student6"),
        Student("student7"),Student("student8"),Student("student9"),
        Student("student10"),
    ]

    # 学生选课
    student_list[0].enroll(course_list[0])
    student_list[0].enroll(course_list[1])
    student_list[1].enroll(course_list[2])
    student_list[1].enroll(course_list[3])
    student_list[2].enroll(course_list[4])
    student_list[2].enroll(course_list[12])
    student_list[2].enroll(course_list[13])
    student_list[2].enroll(course_list[14])
    student_list[3].enroll(course_list[5])
    student_list[3].enroll(course_list[15])
    student_list[3].enroll(course_list[16])
    student_list[4].enroll(course_list[6])
    student_list[4].enroll(course_list[17])
    student_list[4].enroll(course_list[18])
    student_list[4].enroll(course_list[19])
    student_list[5].enroll(course_list[7])
    student_list[6].enroll(course_list[8])
    student_list[7].enroll(course_list[9])
    student_list[8].enroll(course_list[10])
    student_list[9].enroll(course_list[11])

    # 序号2学生多选课程
    student_list[2].enroll(course_list[15])
    student_list[2].enroll(course_list[16])
    student_list[2].enroll(course_list[17])
    student_list[2].enroll(course_list[18])

    # 种群大小和适应度阈值
    population_size_int = 100
    fitness_threshold_float = 50
    # 保留最优的多少个
    retention_number_int=5

    # 获得初始化的种群
    population_list = initialize_population(course_list, timeslots_list, room_list, student_list, population_size_int)

    # 初始化进化次数
    generation_int = 0
    while True:
        # 交叉和变异获得下一代种群
        population_list = genetic_evolution(population_list,retention_number_int)

        # 获取最优个体
```

```python
        best_schedule = max(population_list, key=lambda x: x.fitness())
        best_fitness_float = best_schedule.fitness()

        # 打印当前代的最优适应度
        generation_int += 1
        print(f"Generation {generation_int}: Best Fitness = {best_fitness_float}")
        best_schedule.display()
        print("-" * 40)

        # 如果达到了适应度要求，退出循环
        if best_fitness_float >= fitness_threshold_float:
            break

    print("Final Best Schedule:")
    best_schedule.display()

```

排课模拟数据2024_09_25

示例课程（课程名、任课教师、课时长(单位小时)、优先级(数字越高优先级越高)、不可上课时间)

```python
course_list = [
Course(
       name_str="class1", teacher_str='teacher1', duration_float=1,
       priority_float=1, unavailable_timeslots_list=["time2", "time4"]
       ),
Course("class2", "teacher2", 1, 1, ["time1"]),
Course("class3", "teacher3", 1, 1),
Course("class4", "teacher4", 1, 1, ["time3"]),
Course("class5", "teacher5", 1, 1),
Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
Course("class7", "teacher7", 1, 1),
Course("class8", "teacher8", 1, 1, ["time2"]),
Course("class9", "teacher9", 1, 1),
Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
Course("class11", 'teacher11', 1, 1),
Course("class12", "teacher12", 1, 1, ["time1"]),
Course("class13", "teacher13", 1, 1),
Course("class14", "teacher14", 1, 1, ["time3"]),
Course("class15", "teacher15", 1, 1),
Course("class16", "teacher16", 1, 1, ["time5"]),
Course("class17", "teacher17", 1, 2),
Course("class18", "teacher18", 1, 2, ["time2"]),
Course("class19", "teacher19", 1, 3),
Course("class20", "teacher20", 1, 3, ["time4"]),
    ]
```

示例时间段和教室

```python
# 实例时间段和教室
timeslots_list = ["time1", "time2", "time3", "time4", "time5", "time6"]
room_list = ["Room1", "Room2", "Room3"]
```

示例学生

```python
# 示例学生
student_list = [
    Student("student1"),Student("student2"),Student("student3"),
    Student("student4"),Student("student5"),Student("student6"),
    Student("student7"),Student("student8"),Student("student9"),
    Student("student10"),
    ]
```

示例学生选课

```
# 学生选课
student_list[0].enroll(course_list[0])
student_list[0].enroll(course_list[1])
student_list[1].enroll(course_list[2])
student_list[1].enroll(course_list[3])
student_list[2].enroll(course_list[4])
student_list[2].enroll(course_list[12])
student_list[2].enroll(course_list[13])
student_list[2].enroll(course_list[14])
student_list[3].enroll(course_list[5])
student_list[3].enroll(course_list[15])
student_list[3].enroll(course_list[16])
student_list[4].enroll(course_list[6])
student_list[4].enroll(course_list[17])
student_list[4].enroll(course_list[18])
student_list[4].enroll(course_list[19])
student_list[5].enroll(course_list[7])
student_list[6].enroll(course_list[8])
student_list[7].enroll(course_list[9])
student_list[8].enroll(course_list[10])
student_list[9].enroll(course_list[11])

# 序号2学生多选课程
student_list[2].enroll(course_list[15])
student_list[2].enroll(course_list[16])
student_list[2].enroll(course_list[17])
student_list[2].enroll(course_list[18])
```

模拟结果

种群大小为100、个体变异单个基因的概率为30%，设置适应度大于等于50代表收敛

适应度增加和减少规则：

- 课程时间冲突：同一课程在同一时间在多个教室授课，减少3分适应度
- 学生时间冲突：同一学生在同一时间上多门课程，减少3分适应度
- 不可上课时间：课程安排上课时间为教师不可上课时间，减少3分适应度
- 排课是否均衡：教师未被安排授课，减少5分适应度；教师被安排授课次数大于等于3次，减少2分适应度；排课次数为1-2次，增加1分适应度
- 课程的优先级：安排课程时，适应度增加课程优先级的分数（如课程优先级为3，则增加3分适应度）

以下为排课模拟结果（部分_3代的进化结果）

```
Generation 1: Best Fitness = 3
time1 in Room1: class19 by teacher19 (Priority: 3) | Enrolled: student3, student5
time1 in Room2: class1 by teacher1 (Priority: 1) | Enrolled: student1
time1 in Room3: class4 by teacher4 (Priority: 1) | Enrolled: student2
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class5 by teacher5 (Priority: 1) | Enrolled: student3
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3 | Conflict: student3
Unassigned Teachers: teacher8, teacher3, teacher16, teacher18, teacher17, teacher7
All rooms assigned.
Unassigned Students: student6
----------------------------------------
Generation 2: Best Fitness = 14
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class5 by teacher5 (Priority: 1) | Enrolled: student3
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3 | Conflict: student3
Unassigned Teachers: teacher8, teacher16, teacher18, teacher19
All rooms assigned.
Unassigned Students: student6
----------------------------------------
----------------------------------------
Generation 3: Best Fitness = 17
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class13 by teacher13 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class13 by teacher13 (Priority: 1) | Enrolled: student3
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class15 by teacher15 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class8 by teacher8 (Priority: 1) | Enrolled: student6
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class14 by teacher14 (Priority: 1) | Enrolled: student3
Unassigned Teachers: teacher16, teacher18, teacher5, teacher19
All rooms assigned.
All students enrolled.
----------------------------------------
```

第1代种群最高适应度为3，最高适应度个体基因中有6门课程未进行排课，其中teacher17、teacher18的课程优先级为2，teacher1在time1的不可上课时间进行了排课；在time6中student3出现上课时间冲突，共有1名学生没有排课安排

第2代种群最高适应度为14，最高适应度个体基因中有4门课未进行排课，优先级2的teacher17安排了排课，但优先级为3的teacher19未安排排课，没有教师在不可上课时间安排了排课；在time6出现学生上课时间冲突，共有1名学生没有排课安排

第3代种群最高适应度为17，最高适应度个体基因中有4门课未进行排课，其中teacher18、teacher19的课程优先级为2和3，没有教师在不可上课时间安排了排课；所有学生上课时间无冲突，所有学生安排了排课

.....（省略部分、以下只记录适应度变化的节点）.....

第8代种群最高适应度为28，进化到第24代种群最高适应度为29，到第251最高适应度为32，到2460代最高适应度依然为32

以下为第2460代适应度为32的个体基因（排课安排），因设置的资源配置，必定有两门课无法安排排课

```
----------------------------------------
Generation 2460: Best Fitness = 32
time1 in Room1: class7 by teacher7 (Priority: 1) | Enrolled: student5
time1 in Room2: class3 by teacher3 (Priority: 1) | Enrolled: student2
time1 in Room3: class17 by teacher17 (Priority: 2) | Enrolled: student3, student4
time2 in Room1: class15 by teacher15 (Priority: 1) | Enrolled: student3
time2 in Room2: class10 by teacher10 (Priority: 1) | Enrolled: student8
time2 in Room3: class11 by teacher11 (Priority: 1) | Enrolled: student9
time3 in Room1: class19 by teacher19 (Priority: 3) | Enrolled: student3, student5
time3 in Room2: class12 by teacher12 (Priority: 1) | Enrolled: student10
time3 in Room3: class2 by teacher2 (Priority: 1) | Enrolled: student1
time4 in Room1: class4 by teacher4 (Priority: 1) | Enrolled: student2
time4 in Room2: class5 by teacher5 (Priority: 1) | Enrolled: student3
time4 in Room3: class6 by teacher6 (Priority: 1) | Enrolled: student4
time5 in Room1: class1 by teacher1 (Priority: 1) | Enrolled: student1
time5 in Room2: class13 by teacher13 (Priority: 1) | Enrolled: student3
time5 in Room3: class20 by teacher20 (Priority: 3) | Enrolled: student5
time6 in Room1: class8 by teacher8 (Priority: 1) | Enrolled: student6
time6 in Room2: class9 by teacher9 (Priority: 1) | Enrolled: student7
time6 in Room3: class18 by teacher18 (Priority: 2) | Enrolled: student3, student5
Unassigned Teachers: teacher16, teacher14
All rooms assigned.
All students enrolled.
----------------------------------------
```

第2460代种群最高适应度为32，优先级大于1的课程全部安排排课，未进行排课的teacher16和teacher14的课程优先级为1（默认最低为1）；没有教师上课时间冲突；没有学生上课时间冲突；没有教师排课安排在不可上课时间；从第1代种群到第2460代种群，用时9秒04

结论2024_09_25

实验结果来看，使用遗传算法进行排课可以满足优化排课和快速排课的要求，遗传算法优点在于可以根据需要自由设计适应度计算方法，但需要注意遗传进化算法对适应度计算参数敏感，需要根据需要详细设计适应度计算函数；

具体代码JavaScript版本（基于ChatGPT4o转换，可能有错）

```javascript
// Author: xudawu
// Date: 2024-09-18 16:39:56
// LastEditors: xudawu
// LastEditTime: 2024-09-25 11:49:06

class Course {
    constructor(name, teacher, duration, priority = 1, unavailableTimeslots = []) {
        this.name = name; // 课程名称
        this.teacher = teacher; // 教师名称
        this.duration = duration; // 课程时长
        this.priority = priority; // 课程优先级
        this.unavailableTimeslots = unavailableTimeslots.length > 0 ? unavailableTimeslots : []; // 不可用时间列表
    }
}

class Student {
    constructor(name) {
        this.name = name; // 学生名称
        this.enrolledCourses = []; // 已选课程列表
        this.timeSlots = []; // 学生的上课时间段记录
    }

    enroll(course) {
        this.enrolledCourses.push(course); // 学生选课
    }
}

class Schedule {
    constructor(courseList, timeslotsList, roomList, studentList) {
        this.courseList = courseList; // 课程列表
        this.timeslotsList = timeslotsList; // 时间段列表
        this.roomList = roomList; // 教室列表
        this.studentList = studentList; // 学生列表
        this.scheduleDict = {}; // 存储时段与教室组合的课程安排

        // 初始化安排字典
        for (let timeslot of this.timeslotsList) {
            for (let room of this.roomList) {
                this.scheduleDict[`${timeslot}-${room}`] = null; // 课程初始化为 null
            }
        }
    }

    fitness() {
        let fitnessScore = 0; // 适应度分数
        let courseTimeDict = {}; // 记录课程时间安排
        let teacherAssignedCount = {}; // 记录每位教师被安排的次数

        // 初始化学生的时间冲突列表
        for (let student of this.studentList) {
            student.timeSlots = [];
        }
```

```

        // 初始化冲突信息列表
        this.conflictInfoList = [];

        // 初始化教师安排计数
        for (let course of this.courseList) {
            teacherAssignedCount[course.teacher] = 0;
        }

        // 遍历安排字典
        for (let key in this.scheduleDict) {
            let [timeslot, room] = key.split('-');
            let course = this.scheduleDict[key];

            if (course) {
                // 检测同一课程同一时间在多个教室的冲突
                if (courseTimeDict[`${timeslot}-${course.name}`]) {
                    fitnessScore -= 3; // 发生冲突，降低适应度
                } else {
                    courseTimeDict[`${timeslot}-${course.name}`] = room; // 记录课程安排
                    fitnessScore += course.priority; // 加上课程优先级
                    teacherAssignedCount[course.teacher]++; // 增加教师安排计数

                    // 检测学生时间冲突
                    for (let student of this.studentList) {
                        if (student.enrolledCourses.includes(course)) {
                            if (student.timeSlots.includes(timeslot)) {
                                fitnessScore -= 3; // 冲突，降低适应度
                                this.conflictInfoList.push({ student: student.name, course: course.name, timeslot }); // 记录冲突信息
                            } else {
                                student.timeSlots.push(timeslot); // 更新学生的上课时间
                            }
                        }
                    }
                }

                // 检查教师不可上课时间
                if (course.unavailableTimeslots.includes(timeslot)) {
                    fitnessScore -= 3; // 降低适应度
                }
            }
        }

        // 评估教师安排的均衡性
        for (let [teacher, count] of Object.entries(teacherAssignedCount)) {
            if (count === 0) {
                fitnessScore -= 5; // 教师未被分配
            } else if (count >= 3) {
                fitnessScore -= 2; // 教师安排过多
            } else {
                fitnessScore += 1; // 教师安排均衡
            }
        }
```

```

        return fitnessScore; // 返回适应度分数
    }

    // 基因变异操作
    mutate() {
        const keys = Object.keys(this.scheduleDict); // 获取所有安排的键
        const randomKey = keys[Math.floor(Math.random() * keys.length)]; // 随机选择一个安排
        const randomCourse = this.courseList[Math.floor(Math.random() * this.courseList.length)]; // 随机选择一门课程
        this.scheduleDict[randomKey] = randomCourse; // 替换该位置的课程
    }

    // 基因交叉操作
    crossover(other) {
        const crossoverIndex = Math.floor(Math.random() * Object.keys(this.scheduleDict).length); // 随机选择交叉基因的起点
        const child1Schedule = new Schedule(this.courseList, this.timeslotsList, this.roomList, this.studentList);
        const child2Schedule = new Schedule(this.courseList, this.timeslotsList, this.roomList, this.studentList);
        let index = 0;

        for (let key in this.scheduleDict) {
            if (index <= crossoverIndex) {
                child1Schedule.scheduleDict[key] = this.scheduleDict[key]; // 子代1继承本个体基因
                child2Schedule.scheduleDict[key] = other.scheduleDict[key]; // 子代2继承其他个体基因
            } else {
                child1Schedule.scheduleDict[key] = other.scheduleDict[key]; // 子代1继承其他个体基因
                child2Schedule.scheduleDict[key] = this.scheduleDict[key]; // 子代2继承本个体基因
            }
            index++;
        }

        return [child1Schedule, child2Schedule]; // 返回两个新子代
    }

    // 显示排课结果
    display() {
        let unassignedTeachers = new Set(this.courseList.map(course => course.teacher)); // 未安排教师
        let unassignedRooms = new Set(this.roomList); // 未安排教室
        let unassignedStudents = new Set(this.studentList.map(student => student.name)); // 未安排学生

        for (let key in this.scheduleDict) {
            let [timeslot, room] = key.split('-');
            let course = this.scheduleDict[key];

            if (course) {
                let enrolledStudents = this.studentList.filter(student => student.enrolledCourses.includes(course)).map(student => student.name); // 选课学生列表
```

```
                let enrolledStudentStr = enrolledStudents.length > 0 ? enrolledStudents.join(', ') : "No student enrolled"; // 选课学生字符串
                let teacherUnavailable = course.unavailableTimeslots.includes(timeslot) ? " | Teacher unavailable during this time" : ""; // 教师不可用提示

                // 初始化冲突信息
                let conflictStr = this.conflictInfoList.filter(info => info.course === course.name && info.timeslot === timeslot)
                    .map(info => ` | Conflict: ${info.student}`).join('');

                console.log(`${timeslot} in ${room}: ${course.name} by ${course.teacher} (Priority: ${course.priority}) | Enrolled: ${enrolledStudentStr}${conflictStr}${teacherUnavailable}`);

                // 从未安排列表中移除教师和教室
                unassignedTeachers.delete(course.teacher);
                unassignedRooms.delete(room);
            } else {
                console.log(`${timeslot} in ${room}: Free`);
            }
        }

        // 打印未安排的教师、教室和学生
        console.log("Unassigned Teachers: " + (unassignedTeachers.size > 0 ? Array.from(unassignedTeachers).join(', ') : "All teachers assigned."));
        console.log("Unassigned Rooms: " + (unassignedRooms.size > 0 ? Array.from(unassignedRooms).join(', ') : "All rooms assigned."));
        console.log("Unassigned Students: " + (unassignedStudents.size > 0 ? Array.from(unassignedStudents).join(', ') : "All students enrolled."));
    }
}

// 初始化种群
function initializePopulation(courseList, timeslotsList, roomList, studentList, populationSize = 100) {
    let population = []; // 种群列表初始化
    for (let i = 0; i < populationSize; i++) {
        let schedule = new Schedule(courseList, timeslotsList, roomList, studentList); // 实例化一个个体
        for (let key in schedule.scheduleDict) {
            schedule.scheduleDict[key] = courseList[Math.floor(Math.random() * courseList.length)]; // 随机初始化课程安排
        }
        population.push(schedule); // 加入种群列表
    }
    return population; // 返回初始化的种群列表
}

// 交叉和变异获得下一代种群
function geneticEvolution(population, retentionNumber) {
    population.sort((a, b) => b.fitness() - a.fitness()); // 按适应度排序

    let nextGeneration = []; // 下一代个体列表
    nextGeneration.push(...population.slice(0, retentionNumber)); // 保留最优个体

```

```
    // 选择和交叉，直到达到种群大小
    while (nextGeneration.length < population.length) {
        let parent1 = population[Math.floor(Math.random() * 10)]; // 随机选择父母1
        let parent2 = population[Math.floor(Math.random() * 10)]; // 随机选择父母2
        let [child1, child2] = parent1.crossover(parent2); // 交叉生成两个子代
        nextGeneration.push(child1, child2); // 加入下一代个体列表
    }

    // 子代变异
    for (let child of nextGeneration) {
        if (Math.random() < 0.3) {
            child.mutate(); // 随机变异
        }
    }

    return nextGeneration; // 返回下一代
}

// 主程序入口
const courseList = [
    new Course("class1", 'teacher1', 1, 1, ["time2", "time4"]),
    new Course("class2", "teacher2", 1, 1, ["time1"]),
    new Course("class3", "teacher3", 1, 1),
    new Course("class4", "teacher4", 1, 1, ["time3"]),
    new Course("class5", "teacher5", 1, 1),
    new Course("class6", "teacher6", 1, 1, ["time5", "time6"]),
    new Course("class7", "teacher7", 1, 1),
    new Course("class8", "teacher8", 1, 1, ["time2"]),
    new Course("class9", "teacher9", 1, 1),
    new Course("class10", "teacher10", 1, 1, ["time1", "time6"]),
    new Course("class11", 'teacher11', 1, 1),
    new Course("class12", "teacher12", 1, 1, ["time1"]),
    new Course("class13", "teacher13", 1, 1),
    new Course("class14", "teacher14", 1, 1, ["time3"]),
    new Course("class15", "teacher15", 1, 1),
    new Course("class16", "teacher16", 1, 1, ["time5"]),
    new Course("class17", "teacher17", 1, 2),
    new Course("class18", "teacher18", 1, 2, ["time2"]),
    new Course("class19", "teacher19", 1, 3),
    new Course("class20", "teacher20", 1, 3, ["time4"]),
];

// 实例时间段和教室
const timeslotsList = ["time1", "time2", "time3", "time4", "time5", "time6"];
const roomList = ["Room1", "Room2", "Room3"];

// 示例学生
const studentList = [
    new Student("student1"), new Student("student2"), new Student("student3"),
    new Student("student4"), new Student("student5"), new Student("student6"),
    new Student("student7"), new Student("student8"), new Student("student9"),
```

```
    new Student("student10"),
];

// 学生选课
studentList[0].enroll(courseList[0]);
studentList[0].enroll(courseList[1]);
studentList[1].enroll(courseList[2]);
studentList[1].enroll(courseList[3]);
studentList[2].enroll(courseList[4]);
studentList[2].enroll(courseList[12]);
studentList[2].enroll(courseList[13]);
studentList[2].enroll(courseList[14]);
studentList[3].enroll(courseList[5]);
studentList[3].enroll(courseList[15]);
studentList[3].enroll(courseList[16]);
studentList[4].enroll(courseList[6]);
studentList[4].enroll(courseList[17]);
studentList[4].enroll(courseList[18]);
studentList[4].enroll(courseList[19]);
studentList[5].enroll(courseList[7]);
studentList[6].enroll(courseList[8]);
studentList[7].enroll(courseList[9]);
studentList[8].enroll(courseList[10]);
studentList[9].enroll(courseList[11]);

// 序号2学生多选课程
studentList[2].enroll(courseList[15]);
studentList[2].enroll(courseList[16]);
studentList[2].enroll(courseList[17]);
studentList[2].enroll(courseList[18]);

// 种群大小和适应度阈值
const populationSize = 100; // 种群大小
const fitnessThreshold = 50; // 适应度阈值
const retentionNumber = 5; // 保留最优的个体数量

// 获得初始化的种群
let population = initializePopulation(courseList, timeslotsList, roomList, studentList, populationSize);

// 初始化进化次数
let generation = 0;
while (true) {
    // 交叉和变异获得下一代种群
    population = geneticEvolution(population, retentionNumber);

    // 获取最优个体
    let bestSchedule = population.reduce((best, current) => current.fitness() > best.fitness() ? current : best); // 找到适应度最高的个体
    let bestFitness = bestSchedule.fitness();

    // 打印当前代的最优适应度
    generation++;
    console.log(`Generation ${generation}: Best Fitness = ${bestFitness}`);
    bestSchedule.display();
    console.log("-".repeat(40));

```

```
    // 如果达到了适应度要求，退出循环
    if (bestFitness >= fitnessThreshold) {
        break;
    }
}

console.log("Final Best Schedule:");
bestSchedule.display();

```

排课过程显示在前端

显示当前代和当前代最高排课适应度

可以随时中断排课

点击排课的过程中的代显示当前代具体的排课内容

在排课的时候可视化的显示冲突信息，横坐标为上课教室、纵坐标为上课时间、交点为上课课程和任课教师

随机生成100门课，100个学生，学生随机选择1-5门课，10个上课时间，30间教室，30名教师

课程随机规则

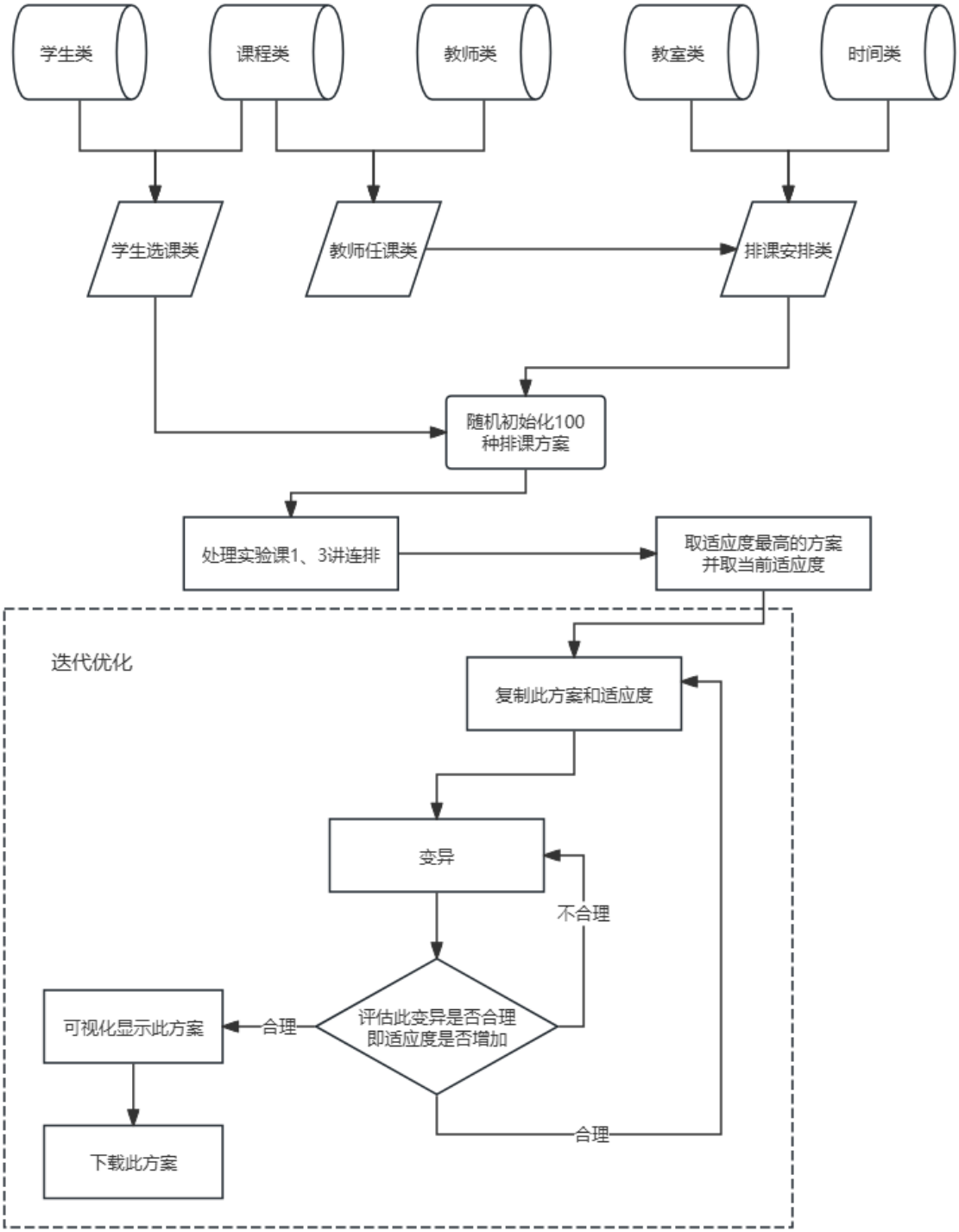
- 课程按照校区、教室类型、课程类型对应的教室类型、时间随机匹配
- 实验课在1、3讲连排两讲，第5讲单排一讲但算排2讲
- 第一代的排课只计算适应度，不处理实验课的特殊排法，之后每一代都会处理实验课，即只有第一代的个体被替代之后，所有的实验课排课才是处理过的

排课优化方向设置

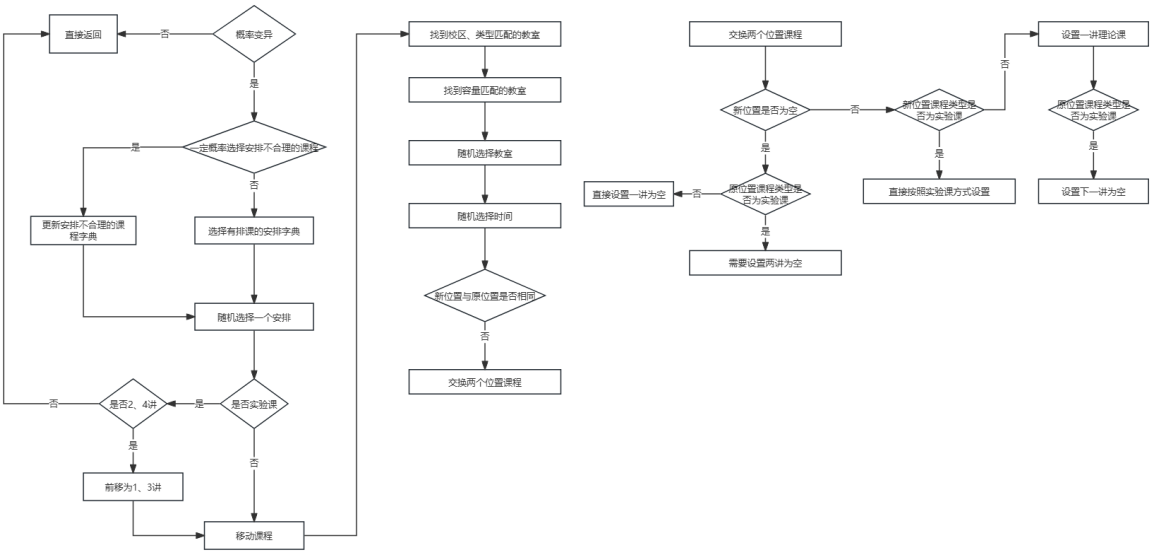
- 使用原本的每一代种群100个方式会有适应度震荡问题，不能稳定朝着适应度增加的方向前进
- 初始化100个，取最好的一个，不再进行基因的交叉，只执行变异，且只选择增加当前适应度的变异，有两种子方式，目前采用第2种，两种方式的代码区别在于固定基因在变异循环内还是循环外，第一种更快，第二种全局更优
    - 1、固定当前基因，变异一次进行判断，适应度增加时，就应用此次变异
    - 2、固定当前基因，进行累积变异，当前变异即使适应度没有增加也依然应用此次变异，直到当前基因累积的变异适应度增加，应用所有累积的变异。
        - 增加变异次数限制：变异超过阈值(50)的次数适应度也无法增加则认为此基因的变异方向不合理，中断此变异路线，重新复制开始的基因执行变异

算法流程图

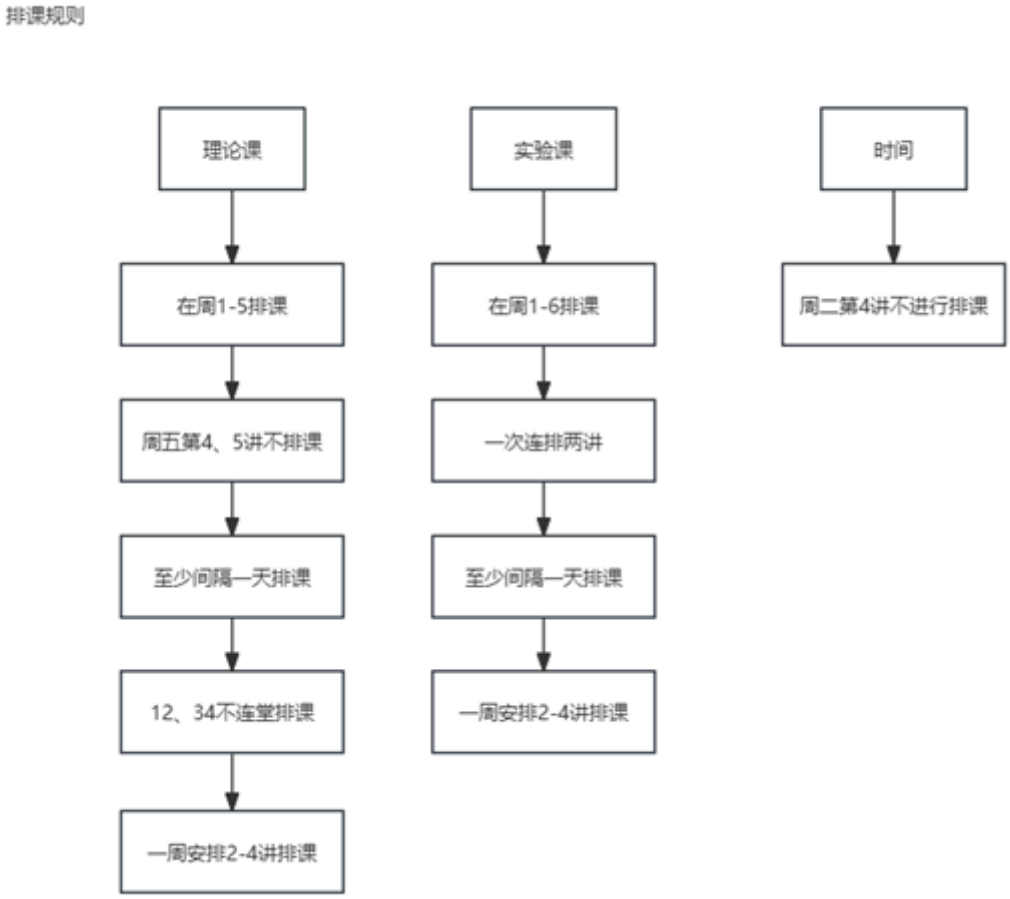
整体流程


学生类
课程类
教师类
教室类
时间类
学生选课类
教师任课类
排课安排类
随机初始化100
种排课方案
处理实验课1、3讲连排
取适应度最高的方案
并取当前适应度
迭代优化
复制此方案和适应度
变异
不合理
评估此变异是否合理
即适应度是否增加
合理
可视化显示此方案
合理
下载此方案

变异

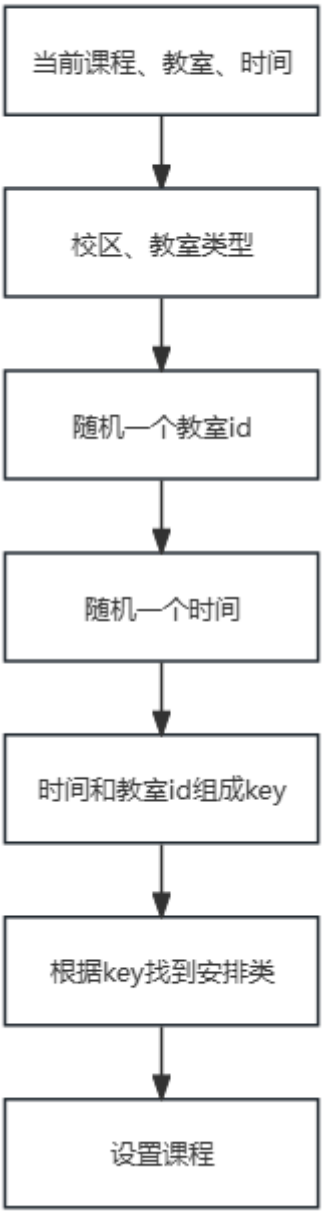


排课规则



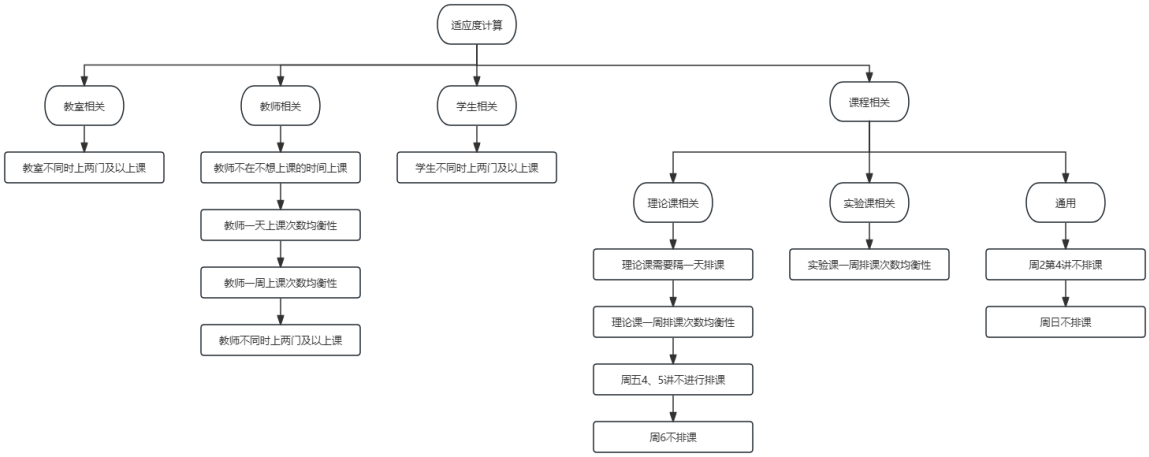
找出可以移动的时间和教室

找出可以移动的时间&教室



适应度设计

加分和减分不应设置差距过大，减少优化过程中陷入谷地局部最优而无法跳出，硬性条件如不允许冲突设置为2分，软性条件设置为1



模拟退火实现排课课表生成

使用深度学习模型实现排课课表生成

六、智能问答系统2024_09_20

使用阿里开源模型qwen2作为自然语言处理的问答模型

使用maxkb搭建知识库

使用docker部署项目发布网页端给用户使用

七、超级查询系统2024_10_15

教务系统目前频繁需要查询和下载各类信息，但缺乏一个统一的查询平台。每次处理业务时都需要定制化开发，既耗时又费力。尽管系统中已经积累了大量数据，但这些数据之间缺乏类似知识图谱的关联性。

业务逻辑

统一查询平台：

- 直接从数据库中获取信息
- 提供两种查询方式，根据关键词查表名和列名得出表数据（更快），通过表内字段查询得出表数据（更全）
- 在表数据中增加筛选功能，进一步筛选出需要的信息

动态显示功能：

- 全库查询一次的时间大概在2-3分钟，这3分钟不允许用户交互系统，空闲的3分钟没有任何用户反馈会大幅降低用户体验
- 牺牲部分性能将查询过程实时显示在前端中，增强用户体验感
- 查询过程中禁用搜索和点击表名功能，防止多次请求消耗服务器算力
- 查询过程结束后，整理所有相关的查询信息，按首字的首字母排序重新显示，便于用户找到需要的信息

用户权限管理：

- 实现用户权限管理，确保不同角色的用户能够访问相应的数据，保护敏感信息的安全。
- 开发人员可以全库查询
- 其余人员根据需要开放不同的表查询权限，同时也能加快查询速度

数据可视化：

- 提供数据可视化功能，帮助用户更直观地分析和理解数据，以支持决策。

八、小工具2024_10_17

Stirling-PDF

pdf处理工具

dockerhub：https://hub.docker.com/r/frooodle/s-pdf

github：https://github.com/Stirling-Tools/Stirling-PDF

docker部署

```
docker pull frooodle/s-pdf
```

启动容器

```
docker run --name s_pdf_container --restart always -p 8080:8080 -d frooodle/s-pdf
```

九、python虚拟环境

激活虚拟环境

```
..\SICAU_JWC_venv_2024_11_15\Scripts\activate
```